

## Experiment: Implementation of Feed Forward Neural Network Using Python

### Aim:

To implement a **Feed Forward Neural Network (FNN)** using Python and train it to perform a simple classification task.

### Objective:

To understand the working of a feed-forward neural network and implement its forward propagation, error calculation, and weight updating using Python.

### Theory

A **Feed Forward Neural Network** consists of an input layer, one or more hidden layers, and an output layer. Information moves only in the forward direction, from the input layer toward the output layer.

The basic steps are:

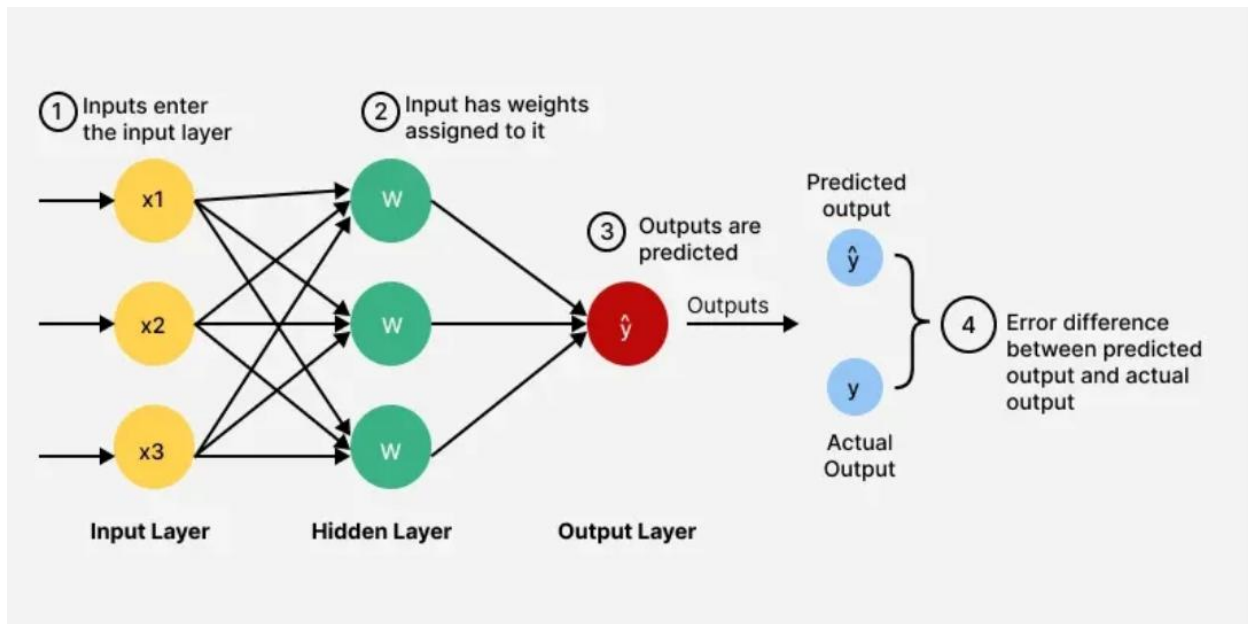
**Input → Hidden Layer → Output Layer → Prediction**

In this program, a small neural network is implemented from scratch using **NumPy**. The **sigmoid activation function** is used, and the network is trained using **backpropagation**.

### Algorithm:

1. Start.
2. Import NumPy.
3. Define the input data and target output.
4. Initialize weights and biases randomly.
5. Perform forward propagation.
6. Calculate the predicted output.
7. Calculate the error between actual and predicted output.
8. Perform backpropagation.
9. Update weights and biases.
10. Repeat the training process for several epochs.
11. Display the final predictions.
12. Stop.

### FLOWCHART:



Code:

```
import numpy as np
```

```
# Sigmoid activation function
```

```
def sigmoid(x):
```

```
    return 1 / (1 + np.exp(-x))
```

```
# Derivative of sigmoid
```

```
def sigmoid_derivative(x):
```

```
    return x * (1 - x)
```

```
# Input data
```

**# XOR problem**

**X = np.array([**

**[0, 0],**

**[0, 1],**

**[1, 0],**

**[1, 1]**

**])**

**# Target output**

**Y = np.array([**

**[0],**

**[1],**

**[1],**

**[0]**

**])**

**# Set random seed**

**np.random.seed(42)**

**# Initialize weights and biases**

**input\_neurons = 2**

**hidden\_neurons = 4**

```
output_neurons = 1
```

```
W1 = np.random.uniform(-1, 1, (input_neurons, hidden_neurons))
```

```
B1 = np.zeros((1, hidden_neurons))
```

```
W2 = np.random.uniform(-1, 1, (hidden_neurons, output_neurons))
```

```
B2 = np.zeros((1, output_neurons))
```

```
# Learning rate
```

```
learning_rate = 0.5
```

```
# Number of training iterations
```

```
epochs = 10000
```

```
# Training
```

```
for epoch in range(epochs):
```

```
    # -----
```

```
    # Forward Propagation
```

```
    # -----
```

```
hidden_input = np.dot(X, W1) + B1
```

```
hidden_output = sigmoid(hidden_input)
```

```
final_input = np.dot(hidden_output, W2) + B2
```

```
predicted_output = sigmoid(final_input)
```

```
# -----
```

```
# Calculate Error
```

```
# -----
```

```
error = Y - predicted_output
```

```
# -----
```

```
# Backpropagation
```

```
# -----
```

```
output_delta = (
```

```
    error *
```

```
    sigmoid_derivative(predicted_output)
```

```
)
```

```
hidden_error = np.dot(output_delta, W2.T)
```

```
hidden_delta = (  
    hidden_error *  
    sigmoid_derivative(hidden_output)  
)
```

```
# -----
```

```
# Update Weights and Biases
```

```
# -----
```

```
W2 += np.dot(hidden_output.T, output_delta) * learning_rate
```

```
B2 += np.sum(output_delta, axis=0, keepdims=True) * learning_rate
```

```
W1 += np.dot(X.T, hidden_delta) * learning_rate
```

```
B1 += np.sum(hidden_delta, axis=0, keepdims=True) * learning_rate
```

```
# Display error
```

```
if epoch % 1000 == 0:
```

```
    loss = np.mean(error ** 2)
```

```
    print("Epoch:", epoch, "Loss:", loss)
```

```
# -----
```

```
# Final Prediction
```

```
# -----
```

```
print("\nFinal Predictions:")
```

```
print("-----")
```

```
for i in range(len(X)):
```

```
    print(
```

```
        X[i],
```

```
        "=>",
```

```
        round(float(predicted_output[i][0]), 4)
```

```
    )
```

```
OUTPUT:
```

```
Epoch: 0 Loss: 0.2629157744191057
Epoch: 1000 Loss: 0.005852947741130165
Epoch: 2000 Loss: 0.0017866014677280703
Epoch: 3000 Loss: 0.00101869497235481
Epoch: 4000 Loss: 0.000704837715931071
Epoch: 5000 Loss: 0.0005360683534415295
Epoch: 6000 Loss: 0.0004312281298020779
Epoch: 7000 Loss: 0.0003599983918529358
Epoch: 8000 Loss: 0.0003085530576490894
Epoch: 9000 Loss: 0.00026970903350004326
```

```
Final Predictions:
```

```
-----
[0 0] => 0.0095
[0 1] => 0.9839
[1 0] => 0.9849
[1 1] => 0.0195
```

### **Result:**

The Feed Forward Neural Network was successfully implemented in Python. The network learned the XOR classification problem through forward propagation and backpropagation.

### **Conclusion**

The experiment demonstrates how a Feed Forward Neural Network processes input data through hidden and output layers. Using backpropagation, the network updates its weights and learns to produce accurate predictions.